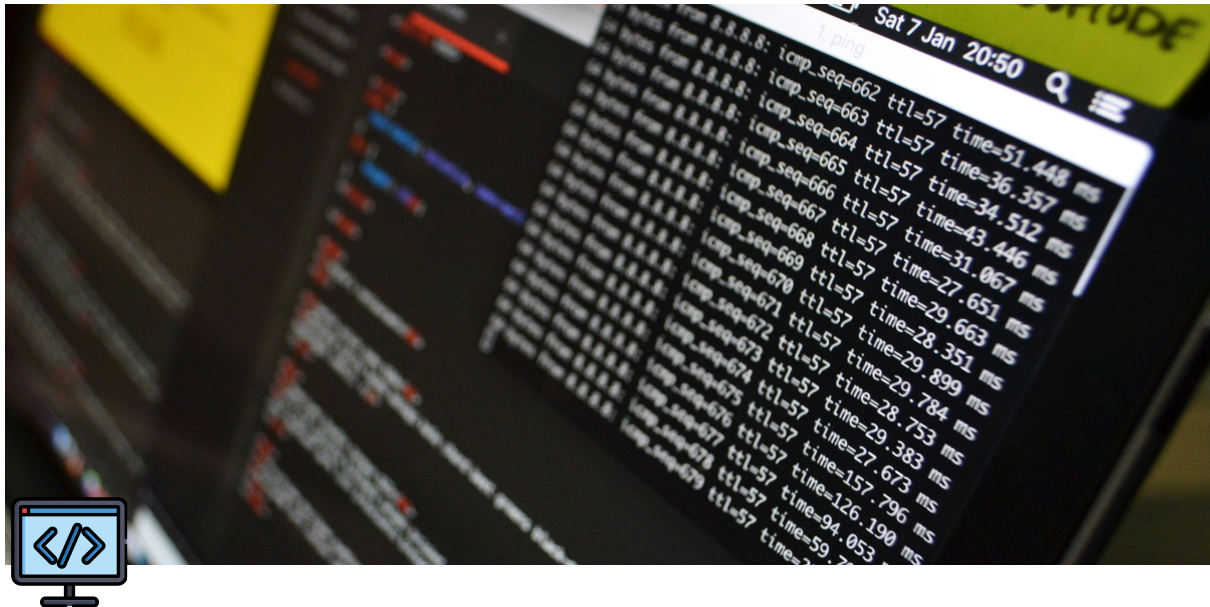




LA BIBLIA PARA APROBAR EL FINAL DE TALLER

🕒 Fecha	@12 de diciembre de 2024 9:39
📖 Materia	Taller de programacion
☰ Area	

Segmentos de memoria

Code Segment:

- solo lectura, contiene el código del programa, instrucciones ejecutables

Data Segment:

- Variables creadas al inicio del programa, existen hasta que este termina
- Pueden ser de acceso global o local

Stack:

- Variables creadas al inicio de una llamada a una funcion y destruidas automaticamete cuando este termina

Heap:

- Variables cuya duracion esta controlada por el programador (run-time)
-

Accesos globales o locales

Acceso Global:

- Una variable o funcion tiene acceso global si se defenine fuera de funciones, metodos o clases
- Estas tienen un likage externo, peuden ser accedidas desde multiples archivos si no se declara como static
- Su tiempo de vida dura hasta que se termine el programa
- Puede causar errores de colisiones de nombres o ser modificadas accidentalmente desde cualquier lugar

Acceso Local:

- Una variable o funcion tiene acceso local si solo puede ser utilizada dentro del bloque de codigo en el que esta definida
- su tiempo de vida esta limitado por el bloque donde fue declarado

Statics

Cuando usar static

- Usar static cuando quiero ocultar variables globales en un archivo especifico
- Conservar estado en funciones locales, por ejemplo contadores
- Definir valores compartidos entre instancias de clases
- Acceso rapido a metodos o constantes de clase sin instancias

Variable Global Static:

- Restringe la visibilidad de la variable a su archivo
- Solo puede ser accedida desde funciones/clases dentro del archivo declarada
- Dura toda la ejecucion del programa

Variable Local Static:

- Se iniciliaza una sola vez, la primera vez que se ejecuta la funcion
- Retiene su valor a lo largo de la ejecucion
- **tiene ambito local pero tiempo de vida global**
- Persiste durante toda la ejecucion del programa

Atributos Static (clase)

- Se comparte entre todas las instancias de la clase
- Pertenece a la clase en general
- Persiste durante toda la ejecucion del programa
- Solo se declara dentro de la clase, pero debe definirse fuera si no se inicializa.

Metodos Static (clase)

- Permite que un metodo de clase sea invocado sin crear una instancia
- No tiene acceso a miembros de la instancia
- Solo puede acceder a miembros estaticos



Namespaces y Using

Se utilizan para organizar y manejar el alcance de identificadores bajo un nombre comun

- evita conflictos de nombres entre diferentes partes

se puede definir un namespace dentro de otro, lo que se conoce como

namespace anidado

Esto permite organizar mejor el código y evitar conflictos de nombres, especialmente cuando tienes múltiples componentes o módulos

```
#include <iostream>

namespace OuterNamespace {
    int outerValue = 10;

    // creo funciones dentro
    void printOuter() {
        std::cout << "Outer namespace value: " << outerValue << "\n";
    }

    // Definición del namespace interior
    namespace InnerNamespace {
        int innerValue = 20;

        void printInner() {
            std::cout << "Inner namespace value: " << innerValue << "\n";
        }
    }
}

int main() {
    // Acceder a los miembros del namespace exterior
    OuterNamespace::printOuter();

    // Acceder a los miembros del namespace interior
    OuterNamespace::InnerNamespace::printInner();
    std::cout << "Inner value directly: " << OuterNamespace::innerValue << "\n";
}
```

```
    return 0;
}
```

Se puede usar `using` para simplificar el acceso a los elementos dentro de un namespace

```
using namespace OuterNameSpace;
```

esto sirve para poder llamar a `printOuter()`; sin tener que agregar el namespace antes

Archivos

- Las funciones `fwrite` y `fread` son funciones de entrada/salida de C, para leer datos en forma de texto o binarios

firmas

- `fwrite (void *ptr, size_t size, size_t count, file *stream)`
- `fread(void *ptr, size_t size, size_t count, file *stream)`
- ambos tienen la misma firma
- `count` es la cantidad de elementos a leer;

abrir y cerrar

- `File *file =fopen(nombre_archivo, permisos y si es binario o no)`
- `fclose(file)`

```
int main() {
    File *file = fopen(nombre_archivo, permisos) //necesita
    FILE *file = fopen("data.bin", "wb");
    if (file == NULL) {
        printf("No se pudo abrir el archivo.\n");
        return 1;
    }
}
```

```

    }

    int arr[] = {1, 2, 3, 4, 5};
    size_t result = fwrite(arr, sizeof(int), 5, file);

    if (result != 5) {
        printf("Error al escribir en el archivo.\n");
    }

    fclose(file);
    return 0;
}

```

Archivos `.h` y `.cpp`

Headers `.h`

- Se utilizan para declarar:
 - Clases
 - Funciones
 - Variables globales
 - Constantes
 - Macros
 - Templates

Todas estas declaraciones sirven como una interfaz que otros archivos puedan incluir y usar, sin conocer los detalles de su implementación

Implementación `.cpp`

- Los archivos contienen la implementación de las declaraciones hechas por los archivos `.h`, se encuentra el cuerpo de los métodos y funciones

- **Compilacion separada:** Cada archivo `.cpp` se compila de forma independiente en un archivo objeto `.o` y luego el enlazador combina todos los archivos para crear el programa final

Ventajas de separacion

- **Modularidad:** El codigo queda mejor organizado
- **Reutilizacion:** Los archivos `.h` actuan como bibliotecas reutilizables, compartir el `.h` con otros proyectos sin exponer los detalles de implementacion
- **Rendimiento compilacion:** Cambiar un `.cpp` solo requiere recompilar ese archivo y no todos los archivos que lo incluyen
- **Encapsulamiento:** Los detalles de implementacion permanecen ocultos en los `.cpp`, los usuarios solo necesitan saber que hacen (declaracion `.h`) y no como (definicion `.cpp`)



FINAL 2019 - TEMA 1

[FINAL 2019C2 - TEMA 1.pdf](#)

- 1) Explique breve y concretamente qué es f:
- ```
int (*f)(short int *, char[3]);
```

Estoy declarando un puntero a una funcion, con 2 tipos de parametros, uno es un puntero a short int y otro un arreglo de char

esto sirve si quiero declarar un puntero f que haga referencia a una funcion

```

int example_function(short int* num, char str[3]){
 (void)str;
 (void)num;
}

int main(){
 int *f(short int*, char[3]) = example_function;

 short int num= 42;
 char[3] array = {'A' , 'B', 'C'};

 int result = f(&num, array);
 std::cout << "Result: " << result << std::endl;
}

```

2) Explique **qué es** cada uno de los siguientes, haciendo referencia a su **inicialización**, su **comportamiento** y el **area de memoria** donde residen:

- a) Una variable **global static**
- b) Una variable **local static**
- c) Un **atributo de clase static**.

|                          | Inicializacion                                                                      | Comportamiento                                                                                                             | Area de memoria |
|--------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------|
| Variable global static   | antes de la ejecucion del main                                                      | Es global en el archivo donde se declara y no puede ser accedida desde otros archivos                                      | data segment    |
| Variable local static    | Se declara dentro de una funcion, y es inicializada la primera vez que es ejecutada | retiene su valor entre multiples llamadas                                                                                  | data segment    |
| Atributo de clase static | No pertenece a ninguna instancia de la clase, sino a la propia clase                | es compartida entre todas las instancias de la clase, si una clase modifica este atributo, el cambio es visible para todas | data segment    |

3) Escribir un programa ISO C que procese el archivo "nros2bytes.dat" sobre sí mismo, duplicando los enteros de 2 bytes múltiplos de 3.

```
void multiplicar_pares(){
 FILE* file = fopen("nros2bytes.dat", "r+");
 if (file == NULL) {
 printf("No se pudo abrir el archivo.\n");
 return -1;
 }

 uint16_t number;

 while (fread(&number, sizeof(uint16_t), 1, file)) {
 if (!(number % 3)) {
 number *= 2;
 fseek(file, - sizeof(uint16_t), SEEK_CUR);
 fwrite(&number, sizeof(uint16_t), 1, file);
 }
 }
 fclose(file);
}
```

```
#include <iostream>
#include <fstream>

void multiplicar_pares() {
 std::ifstream file("nros2bytes.dat", std::ios::in | std::ios::out);
 if (!file.is_open()) {
 std::cerr << "No se pudo abrir el archivo.\n";
 return;
 }

 uint16_t number;

 while (file.read(reinterpret_cast<char*>(&number), sizeof(uint16_t))) {
 if (!(number % 3)) {
 number *= 2;
 file.seekp(-sizeof(uint16_t), std::ios_base::cur);
 file.write(reinterpret_cast<char*>(&number), sizeof(uint16_t));
 }
 }
}
```

```

 file.seekg(-sizeof(number), std::ios::cur);
 file.write(reinterpret_cast<const char*>(&number)
 }
}

file.close();
}

```

4) ¿Cómo se logra que 2 **threads** accedan (lectura/escritura) a un mismo recurso compartido sin que se generen problemas de consistencia? **Ejemplifique.**

Para que 2 threads accedan a un mismo recurso compartido sin que se generen problemas de consistencia se necesitan un mecanismo de bloqueo para que cuando uno acceda al recurso haga esperar a los otros.

Esto en c++ se logra mediante mutex, que es un mecanismo de sincronizacion (**mutual exclusion**)

Mutex permite que solo un thread a la vez acceda a un recurso compartido, al bloquear el mutex, el thread asegura acceso exclusivo, una vez que termina, libera el mutex para que otros threads puedan usar el recurso

C++ incluye una clase RAII, para manejar el mutex, que se llama `lock_guard` y sirve para olvidarnos de hacer un unlock y que no haya errores y que la ejecucion no se congele

- Cada componente que va a manejar recursos compartidos, y por donde distintos threads van operar por ahí, tiene que crear un `std::mutex`
- Cada vez que este componente quiera realizar una operacion sobre el, es necesario que el mutex bloquee y solo deje ejecutar el thread que llego primero, eso lo hacemos con `std::lock_guard<std::mutex> lock(mi_mutex)`

```

int contador;
std::mutex mi_mutex;

```

```

void incrementar(){
 std::lock_guard<std::mutex> lock(mi_mutex);
 ++contador;
}

int main(){

 std::thread t1(incrementar);
 std::thread t2(incrementar);

 t1.join();
 t2.join();

 return 0;
}

```

6) ¿Qué es una **macro** de C? Detalle las **buenas prácticas** para su definición. Ejemplifique

Una macro es una directiva del preprocesador, se suele usar para definir constantes.

Estas se suelen reemplazar directamente en el código fuente antes de la compilación.

Las macros son procesadas por el preprocesador y no ocupan memoria en tiempo de ejecución.

```

#define PI 3.14159
#define MAX_USERS 100

int main() {
 double area = PI * 5 * 5;
 int users = MAX_USERS;
 return 0;
}

```

- No tienen tipo, lo que puede generar errores

7) Describa el **proceso** de transformación de código fuente a un ejecutable. Precise las etapas y las tareas desarrolladas **en cada una de ellas**.

## Preprocesamiento:

Se procesan las directivas del preprocesador, como los include y los define

## Compilacion:

El compilador toma el código fuente preprocesado y lo convierte en código objeto → ensamblador

## Linking (Enlazado)

El enlazador toma uno o mas archivos objetos generados en la etapa anterior, así como las bibliotecas externas, y los combina en un solo archivo ejecutable

8) Indique la **salida** del siguiente programa:

```
class A { A(){cout << "A()" << endl;} ~A(){ cout << "~A()" << endl;} }
class B : public A { B(){cout << "B()" << endl;} ~B(){ cout << "~B()" << endl;} }
int main () { B b; return 0;}
```

```
class A {
 A(){ cout<< "A()" << endl;}
 ~A(){ cout << "~A()" << endl; }
}

class B: public A{

 B(){ cout<< "B()" << endl;}
 ~B(){ cout << "~B()" << endl; }

}

int main() {
 B b;
 return 0;
}
```

El código de salida sería

- A() → se llama al constructor de la clase base primero (en el initialization list)

- B()
- ~B()
- ~A()

9) Implemente una función C++ denominada **Sacar** que reciba dos listas de elementos y devuelva una nueva lista con los elementos de la primera que no están en la segunda:  
`std::list<T> Sacar(std::list<T> a, std::list<T> b);`

```
std::list<T> sacar(std::list<T> a, std::list<T> b){
 std::list<T> result;

 for(const T& item_a: a){
 bool is_a_in_b = false;

 for(const T& item_b: b){
 if(a == b){
 is_a_in_b = true;
 break;
 }
 }

 if(!is_a_in_b) {
 result.push_back(a);
 }
 }

 return result;
}
```

para la comparacion entre elementos

- podria suponer que tienen IDS y tambien un metodo para obtenerlo
- tambien podria suponer que tiene definido el operador por comparacion

==

- 10) Escriba un programa que reciba por **línea de comandos** un **Puerto** y una **IP**. El programa debe establecer una única conexión, quedar en escucha e **imprimir en stdout todo** lo recibido. Al recibir el texto 'FINAL' **debe finalizar** el programa **sin imprimir dicho texto**.

```
#include <arpa/inet.h>
#include <stdio.h>
#include <iostream>
#include <string.h>
#include <sys/socket.h>

#define FINAL_WORD "FINAL"

int main(int argc, char* argv[] {

 if (argc != 3){
 return -1;
 }

 char* ip = argv[1];
 int port = atoi(argv[2]); // paso de char a int

 struct addrinfo hints;
 struct addrinfo result*;

 hints.ai_family = AF_INET; // familia de ip: IPV4
 hints.ai_socktype = SOCK_STREAM; // Protocolo: TCP
 hints.ai_flags = 0; // flags

 int s = getaddrinfo(ip, port, &hints, &result);
 if (s < 0){
 return -1;
 }

 int skt_fd = -1;
 for(struct addrinfo *addr = result; addr != NULL; addr =)
 if (skt_fd != -1){ // quedo un socket distinto abierto
 close(skt_fd);
```

```

 }

 int skt_fd = socket(addr->ai_family, addr->ai_socktype, 0);
 if (skt_fd < 0){
 continue;
 }
 else{
 int s = connect(skt_fd, addr->ai_addr, addr->ai_addrlen);
 if(s < 0){
 continue;
 }
 else{
 break;
 }
 }
}
}

```

```

char final[5] = {'F', 'I', 'N', 'A', 'L'};
std::string msg;
while (true){
 uint8_t c = 0;
 int i = 0;
 s = recv(skt, &c, sizeof(uint8_t), 0);
 if(s < 0){
 return -1;
 }
 while(c == final[i]) {
 msg.push_back(c);
 s = recv(skt, &c, sizeof(uint8_t), 0);
 if(s < 0){
 //error
 }
 i++;
 if(i == 5){

```

```

 return 0;
 }
}
msg.push_back(c);
std::cout << msg;
msg.clear();
}

return 0;
}

```

## Cosas importantes para acordarse

```

// cosas para acordarse
struct addrinfo {
 ai_addrlen;
 ai_addr;
 ai_protocol;
 ai_socktype;
 ai_family;
 ai_next;
 ai_flags;
}

hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_STREAM;
hitns.ai_flags = 0;

getaddrinfo(ip, port, ai_family, ai_sock

int fd = socket(fd, ai_family, ai_sockty

for(addrinfo* addr = result, addr != NUL

int s = connect(fd, ai_addr, ai_addrlen)

send(fd, &var, sizeof(len) ,0)
recv(fd, &var, sizeof(len), 0)

```



# FINAL 2018-2C - TEMA 1

[FINAL 2018C2 - TEMA 1.pdf](#)

Un functor es un objeto que actúa como función, esto se logra sobrecargando el operador de la llamada a la función en una clase

2) ¿Qué es un **functor**? **Ejemplifique.**

- Puede almacenar estado o datos internos

```
// Ejemplo de functor

class Multiply_by{
 int factor;
public:
 Multiply_by(int _factor) : factor(_factor) {}

 int operator()(int x) {
 return x*factor;
 }
}

int main(){

 Multiply_by multiplier(3); // seteo el factor

 std::cout << multiplier(10) << std::endl; // llamo a la f

}
```

1) ¿Por qué las clases que utilizan **templates** se declaran y definen en los **.h**?

Los templates son herramientas que permiten escribir código genérico, los templates permiten que las funciones o clases trabajen con tipos genéricos, lo que elimina la necesidad de duplicar el código para diferentes tipos de datos.

Existen templates:

- clases
- funciones

Estas se declaran en el **.h** debido a la forma que el compilador maneja los templates durante la compilación.

Esto se debe a que los templates no generan código hasta que son instanciados con un tipo específico, por lo tanto el compilador necesita tener acceso tanto a la declaración como a la definición del template al momento de instanciarlo.

- el compilador cuando encuentra un template no genera código de inmediato, espera hasta que se use con un tipo específico.

---

3) Describa con exactitud las siguientes declaraciones/definiciones globales:

- `extern float (*i)[3];`
- `static int *C[3];`
- `static short F(const float *a);`

**Extern float (\*i)[3]**

Esto es un puntero que fue definido fuera del archivo actual, este puntero es un puntero a un array de floats.

**Static int\* C[3]**

Esto declara una variable static local, que puede ser accedida por todos los scopes definidos dentro de ese mismo archivo. Es un puntero a un arreglo de ints.

**Static short F(const float \*a)**

Esto es una declaracion de una funcion que retorna un Short, es un static local, puede ser accedida solo dentro del archivo

la palabra `static` define que es de tipo estatico, lo que significa que persiste durante toda la vida del programa y no es accesible desde otros archivos fuentes.

#### 4) ¿Qué es un Deadlock? Ejemplifique.

Un deadlock pasa cuando el programa utiliza un mecanismo de bloqueo de threads para acceder de forma controlada a un recurso, pero con la particularidad de que al no estar bien implementado estos por alguna razon no liberadoan su acceso de forma correcta , entonces puede darse un bloqueo mutuo

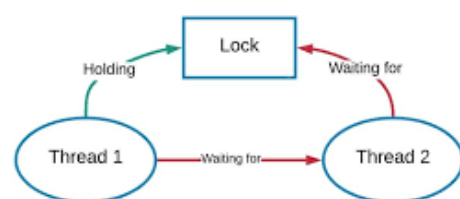


Figure 2: Single Lock Deadlock

Un deadlock es una situacion en la que dos o mas procesos en un sistema estan esperando indefinidamente que uno de los otros libere el recurso necesario para continuar pero ninguno puede proceder porque todos estan bloqueados

5) Explique qué es y para qué sirve una **variable de clase** (o atributo estático) en C++. Mediante un ejemplo de uso, indique cómo se define dicha variable, su inicialización y el acceso a su valor para realizar una impresión simple dentro de un **main**.

Las variable de clase sirven para almacenar informacion que persista para todas las instancias de una misma clase, estas se instancian

- es compartida por todas las instancias
- no requiere que se cree una instancia para acceder a ellas
- se declara usando la palabra clave `static` dentro de la clase



estas se declaran dentro de la clase, pero su definicion e inicializacion se hacen **fuera**

### **Declaración dentro de la clase:**

Indica al compilador que existe una variable estática asociada a la clase. Sin embargo, esto no reserva memoria para la variable.

### **Definición fuera de la clase:**

Reserva la memoria para la variable. Este paso es obligatorio para variables estáticas

---

6) ¿Qué significa que una función es **blocante**? ¿Cómo subsanaría esa limitación en término de mantener el programa 'vivo' ?

Una funcion es bloqueante cuando esta frena la ejecucion del hilo

- una operacion de entrada/salida
- operaciones de sincronizacion como esperar que el mutex sea liberado o un sleep

Para mantener que el programa siga vivo, se podria hacer que esa funcion se ejecute en otro thread, para que ese se encargue de finalizar a la funcion

---

7) Explique qué es y para qué sirve un **constructor MOVE** en C++. Indique cómo se comporta el sistema si éste **no es definido por el desarrollador**.

Un contrstructor **MOVE** en c++ (constructor por movimiento) sirve para rtransferir los recursos de un objeto a otro sin necesidad de copiar los datos e instanciar un objeto nuevo.

En lugar de duplicar los recursos, el constructor move "mueve" los recursos del objeto fuetne al objeto destino, dejando el objeto fuente en un estado valido pero generalmente vacio

- **Eficiencia**: reduce el costo de operacioens de copia, como grandes estructuras de datos

- **Transferencia de propiedad:** Permite que los recursos de un objeto sean transferidos a otros y que la destrucción de los mismos, sea controlada por el nuevo objeto

8) Escribir un programa ISO C que procese el archivo "valuesword.dat" sobre sí mismo, eliminando los words (2 bytes) múltiplos de 16.

```
#define FILE_NAME "valuesword.dat"
#define FILE_TEMP "temp.dat"
#define MULTIPLY_OF 16
int main(){
 uint16_t word;
 File file* = fopen(FILE_NAME, "rb");
 if (!file) {
 return -1;
 }

 File file_temp* = fopen(FILE_TEMP, "wb");
 if (!file_temp) {
 return -1;
 }

 while(fread(&word, sizeof(word), 1, file) == 1){

 if(!(word % MULTIPLY_OF == 0)) {
 fwrite(&word, sizeof(word), 1, file_temp);
 }

 }

 fclose(file);
 fclose(file_temp);

 if(remove(FILENAME) != 0){
 return -1;
 }

 if(rename(FILE_TEMP, FILENAME) != 0) {
```

```

 return -1;
 }

 return 0;

}

```

9) Implemente la función **void ValorHex(char \*hex, int \*ent)** que interprete la cadena **hex** (de símbolos hexadecimales) y guarde el valor correspondiente en el entero indicado por **ent**.

```

void valor_hex(char* hex, int* ent){
 char c;
 // deberia tener en cuenta que ent no puede superar (
 while(*hex != "\0") {
 c = *hex;
 if(c >= 'a' && c <= 'f'){
 *ent = *ent * 16 + (c - "a" + 10);
 }
 else if(c >= 'A' && c <= 'F'){
 *ent = *ent * 16 + (c - "A" + 10);
 }
 else if(c >= '0' && c <= '9'){
 *ent = *ent * 16 + (c - "0");
 }
 else{
 std::cout<< "caracter no valido" << std::endl;
 return;
 }
 hex++;
 }
}

```

10) Implemente una función **C++** denominada **Interseccion** que reciba dos listas de elementos y devuelva una nueva lista con los elementos que se encuentran en ambas listas:

```
std::list<T> Interseccion(std::list<T> a, std::list<T> b);
```

```
// supongo que todos los elementos de a y de b son unicos en
// supongo que los elementos T tienen el operador == definido
std::list<T> interseccion(std::list<T> a, std::list<T> b){
 std::list<T> result;

 for(const T& item_a: a){
 for(const T& item_b: b){
 if(item_a == item_b){
 result.push_back(item_a);
 break;
 }
 }
 }

 return result;
}
```



## FINAL 2024-1C - TEMA x

1) ¿Qué es una Condition Variable? Ejemplifique mediante código.

Condition Variable

2) ¿Que se entiende por RAIL? ¿Por qué se usa? ¿Qué ventajas ofrece? Ejemplifique.

RAIL:

3) ¿Que es un Namespace? ¿Para qué se utiliza? Ejemplifique.

## Namespace

```
using namespace STD
```

• 4) ¿Qué es un iterador de STL? Ejemplifique.

- 5) Declare una clase LISTA para encapsular una lista **doblemente enlazada de elementos genéricos tipo T (template) implementada con memoria dinámica**. Considere:
- Atributos** que necesarios
  - Accesibilidad** a los atributos y métodos de la Clase
  - Incluir constructor **MOVE**
  - Incluir los **operadores** +, ==, << (impresión), >> (carga desde consola), int (cast a int)

6) Escriba las siguientes definiciones/declaraciones:

- Declaración** de un puntero a puntero a Nro de punto flotante, llamado A.
- Definición** de la función MULT, que tome dos enteros con signo y devuelva su producto. Esta función debe ser **visible en todos los módulos del programa**.
- Definición** de un carácter sin signo, solamente visible en el módulo donde se define, llamado C.

- 7) Implemente con **ISO C** la función **int Atrasar(FILE \*f, unsigned OffsetTo, unsigned OffsetFrom, unsigned Legth)** que copie a la posición OffsetTo el contenido del archivo <f> delimitado por <Length> bytes desde <OffsetFrom>. La función debe devolver un entero con el siguiente significado: 0 → OK o <0 → errores. Nota: <OffsetFrom> siempre es menor que <OffsetTo>.

- 8) Escriba un programa que reciba por **línea de comandos** una **IP** y un **Puerto**. El programa **debe conectarse** a la Dirección-Puerto indicada y **quedar a la escucha de paquetes** de datos que terminen con el carácter inicial del mismo (**inician y terminan con el mismo caracter**). El programa deberá **imprimir por stdout los paquetes de largo impar** y **debe finalizar cuando se reciba un paquete de largo 2** (ej.: <>,##,etc.).

• 9) ¿Qué uso tiene el método **notify\_all()**? Ejemplifique.

- 10) Explique y ejemplifique el uso de la directiva de precompilación **#include**

